SIMATS ENGINEERING

CO4 AT1 Class test - 1

NAME : Hitesh C
REG NO : 192524005
COURSE CODE : CSA1401
COURSE NAME : Compiler Design
COURSE FACULTY : AJITHA V

——

Q1. Write down the translation scheme to generate code for assignment statement. List the scheme for generating three address code for the assignment statement x = (a + b) * (c + d)

Answer:

1. Translation Scheme (Syntax-Directed Translation / SDT) for Assignment Statements:

An assignment statement of the form S -> id = E assigns the evaluated value of expression E to identifier id. The translation scheme synthesizes attributes 'place' (holding the variable name or temporary name) and generates appropriate code instructions.

Production Rules and Semantic Actions:

```
S  -> id = E        { emit(id.place '=' E.place) }
E  -> E1 + E2       { E.place = newtemp(); emit(E.place '=' E1.place '+' E2.place)
}
E  -> E1 * E2       { E.place = newtemp(); emit(E.place '=' E1.place '*' E2.place)
}
E  -> ( E1 )        { E.place = E1.place }
E  -> id           { E.place = id.place }
```

2. Three-Address Code (TAC) Generation for x = (a + b) * (c + d):

Using the above translation scheme, the statement is translated step-by-step into intermediate code:

```
t1 = a + b      ; Evaluate left inner expression (a + b)
t2 = c + d      ; Evaluate right inner expression (c + d)
t3 = t1 * t2    ; Multiply temporary results
x = t3          ; Assign final result to target variable x
```

Q2. Consider the following Boolean expression:
(a > b) && (c < d) || (e < f)

The grammar for Boolean expressions is given as:
B -> B1 || B2
B -> B1 && B2
B -> ! B1
B -> E1 relop E2
B -> true
B -> false

(i). Explain how operator precedence and associativity affect the evaluation of the given Boolean expression.
(ii). Construct the Three Address Code (TAC) for the given Boolean expression using short-circuit evaluation.
(iii). Represent the control flow of the Boolean expression using appropriate labels and conditional jump statements.

Answer:

(i) Impact of Operator Precedence and Associativity:

• Precedence: Logical NOT (!) has the highest precedence, followed by relational operators (relop), logical AND (&&), and logical OR (||) with the lowest precedence.

• Expression Grouping: Because && has higher precedence than ||, the expression (a > b) && (c < d) || (e < f) is grouped as ((a > b) && (c < d)) || (e < f).

• Associativity: Both && and || associate from left to right.

• Short-Circuit Evaluation: In short-circuit evaluation, evaluation stops as soon as the truth value is determined:

1. If (a > b) is false, (a > b) && (c < d) immediately evaluates to false without testing (c < d). Evaluation skips to check the right operand of ||, which is (e < f).
2. If (a > b) is true, (c < d) is checked. If (c < d) is also true, the whole expression evaluates to true, and (e < f) is completely skipped.

(ii) & (iii) Three Address Code (TAC) with Short-Circuit Control Flow:

Let L_true be the target label when the Boolean expression is TRUE, and L_false be the target label when it is FALSE:

```
100: if a > b goto 102   ; Check (a > b); if true, test second AND condition
101: goto 104            ; False: short-circuit AND, jump to check OR term (e < f)
102: if c < d goto L_true; True: AND term is true; short-circuit OR, jump to
L_true
103: goto 104            ; False: jump to check OR term (e < f)
104: if e < f goto L_true; Check (e < f); if true, jump to L_true
105: goto L_false        ; False: both branches failed, jump to L_false
106: L_true: ...         ; Target code when expression evaluates to TRUE
107: L_false: ...        ; Target code when expression evaluates to FALSE
```

Q3. Consider the following assignment statements involving pointers:

x = *y;

a = &x;

(i). Construct the Three Address Code (TAC) for the above statements. Clearly represent the semantics of:

(a) Pointer dereferencing operation

(b) Address-of operator

(ii). Represent the generated intermediate code in the following forms:

(a) Quadruples (op, arg1, arg2, result)

(b) Triples (index, op, arg1, arg2)

(c) Indirect Triples (pointer-based representation)

Answer:

(i) Three Address Code (TAC) & Semantics of Pointer Operations:

```
1. x = *y    ; Pointer dereference assignment
2. a = &x    ; Address-of assignment
```

• (a) Pointer Dereferencing (x = *y): The contents of pointer variable y represent a memory address. The dereference operator '*' fetches the value stored at address y and assigns it to x. In TAC, '*y' on the RHS represents an R-value pointer dereference.

• (b) Address-of Operator (a = &x): Retrieves the memory location/address of variable x and assigns that address value to pointer variable a. In TAC, '&x' on the RHS represents address extraction.

(ii) Intermediate Code Representations:

(a) Quadruples Representation (op, arg1, arg2, result):

Index	op	arg1	arg2	result
(0)	=*	y	—	x
(1)	=&	x	—	a

(b) Triples Representation (index, op, arg1, arg2):

Index	op	arg1	arg2
(0)	*	y	—
(1)	=	x	(0)
(2)	&	x	—
(3)	=	a	(2)

(c) Indirect Triples Representation:

Indirect Triples maintain an array of pointers (Statement List) pointing to entries in the Triples table. This permits reordering of statements during optimization without modifying triple references.

Statement List Array:

Statement Index	Triple Pointer
statement[0]	(0)
statement[1]	(1)
statement[2]	(2)
statement[3]	(3)

Q4. Consider the following program fragments:

(i). while (A < C && B > D) do
if (A == 1) then
C = C + 1
else
while (A <= D) do
A = A + B

Generate the Three Address Code (TAC) for the above program fragment. Clearly show:
(i) Conditional and unconditional jump statements
(ii) Use of labels for control flow
(iii) Evaluation of Boolean expressions

(ii). main()
{
int i;
int a[10];
i = 1;
while (i <= 10)
{
a[i] = 0;
i = i + 1;
}

}

Translate the executable statements of the above C program into Three Address Code (TAC). Clearly indicate:
(i) Representation of array elements
(ii) Loop control using labels
(iii) Increment and assignment operations

Answer:

Part (i) Three Address Code (TAC) Generation for Control Structures:

```
L1: if A < C goto L2       ; Outer while condition 1: if true, test condition 2
    goto L_exit            ; Short-circuit false: exit outer loop
L2: if B > D goto L3       ; Outer while condition 2: if true, enter body
    goto L_exit            ; False: exit outer loop
L3: if A == 1 goto L4      ; If condition check: if true, jump to Then branch
    goto L5                ; False: jump to Else branch (inner loop)
L4: t1 = C + 1             ; Then branch: C = C + 1
    C = t1
    goto L1                ; Loop back to outer while header
L5: if A <= D goto L6      ; Inner while loop condition test
    goto L1                ; Inner loop finished: return to outer while header
L6: t2 = A + B             ; Inner while body: A = A + B
    A = t2
    goto L5                ; Repeat inner while loop test
L_exit:                    ; Target exit point of outer loop
```

• Explanation of Features:
- (i) Jump Statements: Conditional jumps (if A < C goto L2, if A == 1 goto L4, etc.) evaluate branch decisions; unconditional jumps (goto L1, goto L5, goto L_exit) enforce loop iteration and structural transfers.
- (ii) Labels: L1 (outer loop check), L2 (second condition check), L3 (outer body entry), L4 (then block), L5 (inner loop header), L6 (inner body), L_exit (loop termination).

- (iii) Boolean Short-Circuiting: Expression A < C && B > D evaluates left-to-right; if A < C fails, execution immediately jumps to L_exit without testing B > D.

Part (ii) TAC for C Program (Array & Loop Execution):

Array Address Calculation Assumption: Each integer element occupies 4 bytes (element width = 4 bytes). The offset of element a[i] is computed as i * 4.

```
100: i = 1                ; Initialize loop counter i = 1
101: L_while: if i <= 10 goto 103  ; Loop condition check: if i <= 10, proceed
102: goto L_end           ; Loop condition false: exit loop
103: t1 = i * 4           ; Calculate byte offset: t1 = i * 4 bytes
104: a[t1] = 0            ; Assign 0 to array element at calculated offset
105: t2 = i + 1           ; Increment i by 1 (t2 = i + 1)
106: i = t2               ; Update counter variable i
107: goto L_while         ; Unconditional jump back to loop header
108: L_end:               ; Loop termination label
```

• Explanation of Key Components:

- (i) Array Representation: t1 = i * 4 computes relative byte offset from base address of array a. a[t1] = 0 stores 0 into the element at index i.
- (ii) Loop Control: L_while marks the loop header; conditional branch at 101 checks termination condition i <= 10; goto L_while at 107 repeats the loop.
- (iii) Increment & Assignment: t2 = i + 1 followed by i = t2 updates counter; a[t1] = 0 performs indexed store.

Q5. Consider the following Boolean expression:
(a < b) || (c < d) && (e < f)

Using the concept of backpatching, answer the following:
(i) Write the syntax-directed definition (SDD) / translation scheme for Boolean expressions based on the grammar:
B -> B1 || B2
B -> B1 && B2
B -> ! B1
B -> E1 relop E2
B -> true

B -> false
(ii) Generate the Three Address Code (TAC) for the given Boolean expression using short-circuit evaluation.
(iii) Clearly show the following during translation:
• Construction of truelist and falselist
• Use of makelist, merge, and backpatch functions
• Final labeled intermediate code after backpatching

Answer:

(i) Syntax-Directed Definition (SDD) for Backpatching:

Backpatching synthesizes attributes truelist and falselist (lists of jump instructions needing target location binding) using auxiliary functions:

• makelist(i): Creates a new list containing quad index i.
• merge(p1, p2): Concatenates list p1 and list p2.
• backpatch(p, i): Inserts target label/quad i into all instructions in list p.
• nextquad: Returns index of the next instruction to be emitted.

SDD Semantic Rules:

```
1. B -> B1 || M B2  { backpatch(B1.falselist, M.quad);
                      B.truelist = merge(B1.truelist, B2.truelist);
                      B.falselist = B2.falselist; }

2. B -> B1 && M B2  { backpatch(B1.truelist, M.quad);
                      B.truelist = B2.truelist;
                      B.falselist = merge(B1.falselist, B2.falselist); }

3. B -> ! B1        { B.truelist = B1.falselist;
                      B.falselist = B1.truelist; }

4. B -> E1 relop E2 { B.truelist = makelist(nextquad);
                      B.falselist = makelist(nextquad + 1);
                      emit('if' E1.val relop.op E2.val 'goto _');
                      emit('goto _'); }

5. M -> epsilon     { M.quad = nextquad; }
```

(ii) & (iii) Step-by-Step Backpatching Translation of (a < b) || (c < d) && (e < f):

Parsing Order (&& takes precedence over ||): (a < b) || ((c < d) && (e < f))

Let B1 = (a < b), B2 = (c < d), B3 = (e < f), B23 = B2 && B3, and B = B1 || B23.

Step 1: Evaluate B1 = (a < b) starting at Quad 100:

```
100: if a < b goto _
101: goto _
```

Attributes created:
B1.truelist = [100]
B1.falselist = [101]

Step 2: Process OR marker M1 after B1:

M1.quad = 102.
Semantic Action: backpatch(B1.falselist, 102) -> Instruction 101 becomes '101: goto 102'.

Step 3: Evaluate B2 = (c < d) starting at Quad 102:

```
102: if c < d goto _
103: goto _
```

Attributes created:
B2.truelist = [102]
B2.falselist = [103]

Step 4: Process AND marker M2 after B2:

M2.quad = 104.
Semantic Action: backpatch(B2.truelist, 104) -> Instruction 102 becomes '102: if c < d goto 104'.

Step 5: Evaluate B3 = (e < f) starting at Quad 104:

```
104: if e < f goto _
105: goto _
```

Attributes created:
B3.truelist = [104]
B3.falselist = [105]

Step 6: Synthesize attributes for B23 = B2 && B3:

• B23.truelist = B3.truelist = [104]
• B23.falselist = merge(B2.falselist, B3.falselist) = merge([103], [105]) = [103, 105]

Step 7: Synthesize attributes for complete expression B = B1 || B23:

• B.truelist = merge(B1.truelist, B23.truelist) = merge([100], [104]) = [100, 104]
• B.falselist = B23.falselist = [103, 105]

Step 8: Final Backpatching to True Target (L_true / Quad 106) & False Target (L_false / Quad 107):

• backpatch(B.truelist, L_true) -> Fills instructions 100 and 104 with goto L_true.
• backpatch(B.falselist, L_false) -> Fills instructions 103 and 105 with goto L_false.

Final Labeled Intermediate Code after Backpatching:

```
100: if a < b goto L_true    ; Backpatched from B1.truelist
101: goto 102                ; Backpatched from B1.falselist to M1.quad
102: if c < d goto 104       ; Backpatched from B2.truelist to M2.quad
103: goto L_false            ; Backpatched from B2.falselist
104: if e < f goto L_true    ; Backpatched from B3.truelist
105: goto L_false            ; Backpatched from B3.falselist
106: L_true: ...             ; Execution path when Boolean expression is TRUE
107: L_false: ...            ; Execution path when Boolean expression is FALSE
```